Manejo Caatingueiro — Back end

Documentação técnica da API (Spring Boot, Spring Data JPA, Lombok, SLF4J, Jakarta). Base: Especificação de Requisitos — Revisão 3 e o prompt de back end do projeto.

Estado atual do código

O código foi escrito conforme o prompt, mas **ainda não foi compilado nem testado com Maven**: o ambiente em que foi gerado não alcançava o Maven Central. Foi feita apenas uma checagem de sintaxe com javac em todos os arquivos (sem erros de sintaxe; os demais erros do javac são apenas dependências ausentes).

Foram escritos 122 arquivos Java de produção e 12 de teste (50 casos de teste), cobrindo cada regra de negócio. Nenhum foi executado. Espere pequenos ajustes no primeiro `mvn clean verify`.

1. Visão geral

A API entrega o núcleo do produto: o **painel de chuva hiperlocal** (chuva acumulada vs. normal da localidade, com fonte e data de atualização) e a **sugestão consultiva de plantio** para milho, mamona e feijão. A sugestão é sempre uma recomendação, nunca uma determinação (RN12): nenhum endpoint bloqueia ou penaliza uma decisão divergente, e toda mensagem devolve a decisão ao agricultor.

- **Stack:** Java 21, Maven, Spring Boot 3.2.5, Spring Web, Spring Data JPA, Lombok, Validation, Spring Security (JWT), MySQL (H2 só em testes), springdoc.
- **Jakarta:** todo o código usa `jakarta.*` (persistence, validation, servlet, annotation), padrão do Spring Boot 3. O único `javax` é `javax.crypto.SecretKey`, tipo do JDK exigido pela jjwt.
- **Dependências extras (justificadas):** jjwt 0.12.5 (JWT) e springdoc-openapi 2.3.0 (Swagger).
- **Fora do escopo (Pós-MVP):** solicitação de apoio hídrico/carro-pipa (RF09, RF10, RF11, RN03, RN05, RN06, RN14).

2. Arquitetura em camadas

Fluxo **Controller** → **Service** → **Repository** → **Banco**, com a Entity trafegando entre as camadas. Controllers só validam a entrada e delegam; todas as regras de negócio ficam nos Services; repositórios estendem `JpaRepository` (sem DAO). Services têm interface `XxxIService` e implementação `XxxService`; injeção só por construtor (`@RequiredArgsConstructor`); `@Slf4j` em todos os Services e no handler de exceções; nenhum dado pessoal vai para o log. As relações são LAZY, com `@EntityGraph` nas leituras que o controller serializa.

Pacote (br.com.ifba.…)	Conteúdo
infrastructure	PersistenceEntity (id, criadoEm, atualizadoEm), exceções, GlobalExceptionHandler (ApiError em português), SecurityConfig, JwtService, propriedades, ETag, seeders
usuario, auth	RF01 cadastro e RF02 login com JWT; perfis AGRICULTOR, LIDERANCA_COMUNITARIA, GESTOR_PUBLICO
localidade, cultura	Município → povoado; culturas com critério e limiar (padrão com fonte e limiar do usuário), RF05
precipitacao, clima	RF03 chuva manual; RF18 ingestão (FonteDadosClimaticosService, stub, job desligado por padrão)
painel	RF04 painel de chuva e RF06 sugestão consultiva, com auditoria em SugestaoPlantio
reservatorio, sync	RF07 nível, RF08 monitor agregado, RF17 sincronização idempotente em lote
alerta	RF12 emissão (liderança/gestor) e RF13 consulta (RN07)
mural, checklist	RF14/RF15 e RF16, isolados e desligados por feature flag

3. Diagrama de classes (modelo de domínio)

Gerado a partir das entidades JPA implementadas. Em azul, o núcleo do MVP; em âmbar e tracejado, os módulos atrás de feature flag. Enums usados nos atributos: Perfil, Abastecimento, TipoLocalidade, TipoCriterio, FontePrecipitacao, NivelSugestao, TipoReservatorio, EscalaNivel, TipoAlerta, Severidade.

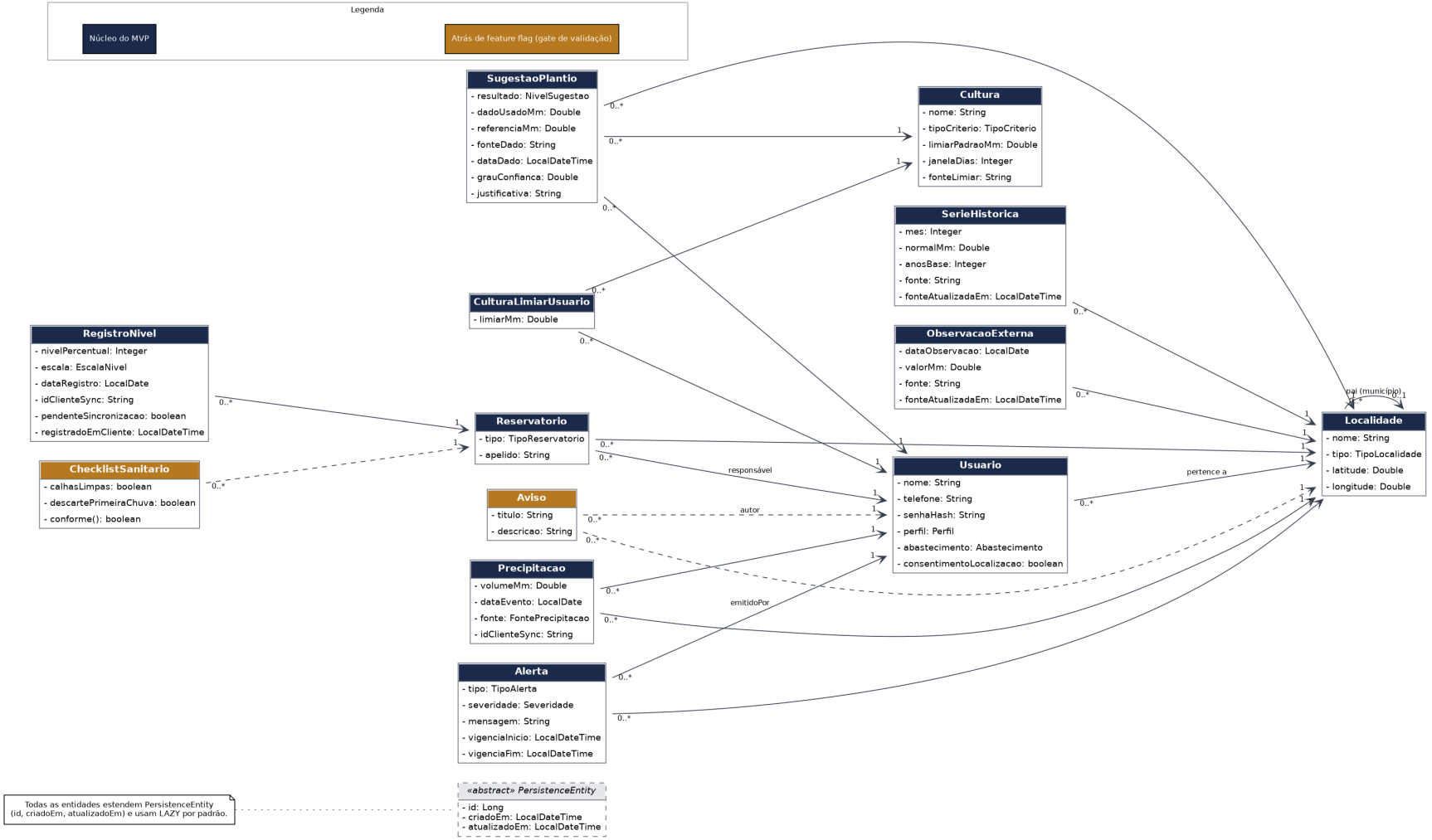


Figura 1 — Entidades, atributos e relacionamentos (multiplicidades nas pontas). Código-fonte do diagrama em docs/classes.dot.

4. Endpoints (/api/v1)

RF	Endpoint	Quem acessa
RF01	POST /usuarios	Público para AGRICULTOR; outros perfis exigem token de gestor
RF02	POST /auth/login	Público
—	GET /usuarios/me; POST, GET /localidades; GET /localidades/{id}	Autenticado (POST de localidade: gestor)
RF05	GET, POST, PUT /culturas; PUT /culturas/{id}/limiar-usuario	Autenticado (POST/PUT: gestor; limiar próprio: qualquer um)
RF03	POST, GET /precipitacoes	Autenticado; a listagem traz só os próprios registros
RF04	GET /localidades/{id}/painel-chuva?cultura=	Autenticado; ETag e Cache-Control
RF06	GET /localidades/{id}/sugestao-plantio?cultura=	Autenticado; registra a sugestão emitida (sem cache)
RF07	POST, GET /reservatorios; POST /reservatorios/{id}/niveis	Autenticado; oculto a quem usa só água encanada
RF08	GET /localidades/{id}/monitor-reservatorios	Autenticado, da própria localidade; só agregado (RN15)
RF12/13	POST /alertas; GET /alertas?localidadeId=	Emissão: liderança ou gestor; consulta: usuários da localidade
RF17	POST /sync/registros	Autenticado; lote de até 200 itens, idempotente
RF18	POST /admin/ingestao/localidades/{id}; POST /admin/ingestao/todas	GESTOR_PUBLICO
RF14/15	POST, GET /avisos	Só com app.features.mural=true
RF16	POST /reservatorios/{id}/checklists; GET .../ultimo	Só com app.features.checklist=true

5. Regras de negócio → implementação → teste

RN	Implementação	Teste
RN01	ReservatorioService.saveNivel: só o responsável registra o nível	ReservatorioServiceTest
RN02	Um registro por reservatório por dia, o novo substitui; constraint única; conflito de sync decidido pelo horário do cliente	ReservatorioServiceTest, RegistroNivelRepositoryTest
RN04	PainelChuvaService.historicoSuficiente; SugestaoPlantioService devolve DADOS_INSUFICIENTES	PainelChuvaServiceTest, SugestaoPlantioServiceTest
RN07	AlertaService + LocalidadeService.podeAcessar	AlertaServiceTest
RN09	CulturaService (limiar padrão com fonte; limiar do usuário sem bloqueio)	SugestaoPlantioServiceTest
RN10	findUltimosAgregaveis considera só registros sincronizados	RegistroNivelRepositoryTest
RN12	Resultados não binários; toda mensagem devolve a decisão ao agricultor	SugestaoPlantioServiceTest
RN13	ETag (EtagConfig), Cache-Control nos controllers, sync idempotente por idClienteSync	PainelControllerTest, SyncServiceTest
RN15	MonitorService: mínimo de responsáveis distintos, com consentimento, só agregado	MonitorServiceTest
RN16	PainelDto e SugestaoDto sempre com fonte e data de atualização	PainelChuvaServiceTest, PainelControllerTest

6. Suposições registradas (precisam de validação)

O prompt não especificava estes pontos. Foi escolhida a opção mais simples, configurável quando possível:

#	Suposição	Onde mudar
1	Limiares de chuva do perfil dev (milho 120 mm, mamona 100 mm, feijão 150 mm) são **placeholders sem base agronômica**, rotulados como tal. Validar com técnico agrícola ou extensionista.	DevDataSeeder; PUT /culturas
2	Faixas da sugestão: razão chuva/limiar ≥ 1 favorável; entre 0,6 e 1 atenção; abaixo de 0,6 considerar alternativa. Não é critério agronômico validado.	app.sugestao.faixa-atencao
3	Série suficiente (RN04): ao menos 10 anos de base na normal do mês, da própria localidade ou do município.	app.clima.anos-minimos-serie
4	Grau de confiança = 0,6 × min(1, anosBase/30) + (dado externo 0,4; só manual 0,2). Heurística simples.	SugestaoPlantioService
5	Estação chuvosa começa em novembro. A validar para Irecê.	app.clima.inicio-estacao-chuvosa-mes
6	Dado externo é a base; registro manual só entra quando não há dado externo no período.	PainelChuvaService.medirPeriodo
7	Escala do reservatório: ≥75% cheio, ≥40% médio, ≥15% baixo, abaixo disso crítico.	ReservatorioService.escalaDe
8	RN15: mínimo de 5 responsáveis distintos; consentimento cobre localização e uso agregado dos dados; o agregado do município inclui os povoados.	app.monitor.minimo-participantes
9	Cadastro público cria só AGRICULTOR; liderança e gestor são criados por um gestor. O primeiro gestor vem das variáveis BOOTSTRAP_*. /admin/** é só do gestor (não há perfil ADMIN).	UsuarioService; SecurityConfig
10	RN10: o servidor só recebe registros já sincronizados; a flag de pendência existe e é respeitada nas consultas, mas a fila vive no app.	RegistroNivel.pendenteSincronizacao
11	Login por telefone (identificador único), sem recuperação de senha.	AuthService

7. Fonte de dados climáticos (RF18): o que falta

A interface `FonteDadosClimaticosService` é o ponto de troca; o sistema não conhece INMET, Cemaden ou CHIRPS. O padrão é o **stub sintético** (dados fictícios, nada representa chuva real). Para ligar uma fonte real:

- Concluir o spike técnico: avaliar série histórica e chuva recente para os povoados-alvo.
- Criar uma classe @Component com @ConditionalOnProperty(app.clima.fonte=<nome>) que implemente nomeFonte(), serieHistorica(...) e chuvaRecente(...), mapeando município/povoado para estação ou grade.
- Definir CLIMA_FONTE=<nome> e, após validar, CLIMA_INGESTAO_AGENDADA=true.
- Revisar app.clima.anos-minimos-serie conforme o que a fonte realmente oferece.

8. Como rodar

Variável	Obrigatória	Uso
DB_URL, DB_USER, DB_PASSWORD	sim	MySQL (nunca fixos no código)
JWT_SECRET	sim	Assinatura do JWT; mínimo de 32 caracteres
JWT_EXPIRACAO_MIN	não (480)	Validade do token
JPA_DDL_AUTO	não (update)	Em produção, usar validate com Flyway ou Liquibase
CLIMA_FONTE, CLIMA_INGESTAO_AGENDADA	não	Fonte de dados e job de ingestão
BOOTSTRAP_GESTOR_TELEFONE, BOOTSTRAP_GESTOR_SENHA	não	Cria o primeiro gestor, se ainda não houver nenhum

Comandos: `mvn clean verify` (compila e testa), depois `mvn spring-boot:run -Dspring-boot.run.profiles=dev`. Swagger em `/swagger-ui.html`. O perfil dev carrega localidades,

culturas e dados climáticos **fictícios**; não usar em produção.

9. Pendências e riscos conhecidos

• **Compilação e testes nunca executados.** É o primeiro passo antes de qualquer outra coisa.

• **Gate de validação:** mural, checklist, alertas e monitor não foram validados em campo. Mural e checklist ficam desligados por padrão; as tabelas existem mesmo com a flag desligada.

• **RN11** (checklist não conforme sinalizado no monitor) não está integrada, para manter o módulo isolado.

• **Alertas derivados do painel** (chuva abaixo do esperado, início das chuvas) não são gerados automaticamente; a emissão é manual (RF12).

• **Login sem limite de tentativas**: falta proteção contra força bruta.

• **LGPD:** a planilha da pesquisa traz nome, telefone e e-mail reais e nunca deve ser versionada (o .gitignore bloqueia *.xlsx). O código não registra telefone, senha nem token em log.

• **Offline-first no servidor** está fora do escopo: há cache (ETag e Cache-Control) e fila de escrita idempotente.